

Отчёт по оптимизации кода карты Godot-проекта

1. Короткий вывод

Код в целом написан **хорошо для текущей стадии проекта**.

Это не наивная карта, где каждая клетка является отдельной `Node2D`. Уже есть правильная база:

- + тайловый стриминг;
- + LOD по зуму;
- + фоновые потоки для тяжёлых операций;
- + дисковый кэш;
- + bundle-кэш;
- + запечённые слои;
- + разделение данных и рендера;
- + отсутствие Node на каждую клетку;
- + Python-пайплайн для офлайн-геоданных.

Главное уточнение:

`IrregularCellProvider` с `for cell in _cells` и `raster_px = 1024` – не критическая проблема, если этот слой используется как `tool/debug/bake`-слой, а не как постоянный `runtime`-слой игры.

То есть прошлые опасения по этим двум пунктам надо понизить.

С учётом этого итоговая оценка такая:

Архитектура карты: 8/10
Оптимизация текущего прототипа: 7.5/10
Готовность к большой карте после запекания слоёв: 8/10
Готовность к полной игре с экономикой/AI/логистикой: 6.5/10, потому что симуляционное ядро ещё не построено

Главный вывод:

Переписывать всё не надо.
Переходить на C# сейчас не надо.
Нужно точно укрепить `runtime`-часть: кэш, обновление тайлов, бюджет создания текстур, разделение `bake/debug` и `game runtime`.

2. Что сделано правильно

2.1. Карта сделана через тайлы, а не одной огромной картинкой

Это правильное фундаментальное решение.

В `TileMapView.gd` активные тайлы хранятся так:

```
var _active: Dictionary = {}
```

То есть в сцене живут только видимые тайлы, а не вся планета сразу.

Правильно:

```
видимая область камеры -> нужные тайлы -> Sprite2D только для них
```

Неправильно было бы:

```
20 000 клеток = 20 000 Node2D / Polygon2D / Area2D
```

У тебя этого нет. Это большой плюс.

2.2. Есть LOD

Уровень детализации выбирается по зуму камеры:

```
var lod_f := log(WORLD_PX * lod_zoom / TILE_PX) / log(2.0)
```

Также есть гистерезис:

```
const LOD_HYSTERESIS := 0.15
```

Это хорошо, потому что LOD не должен прыгать туда-сюда при малейшем движении колеса.

2.3. Есть ограничение создания тайлов за кадр

```
const MAX_GEN_PER_FRAME := 16
```

Это правильная защита от резкого фриза при зуме или быстром перемещении камеры.

Но есть нюанс: этот лимит ограничивает **создание спрайтов**, но не всегда ограничивает количество запущенных запросов/фоновых задач. Об этом ниже.

2.4. Декодирование и тяжёлый рендер вынесены в WorkerThreadPool

Это хорошо сделано в:

```
OnlineTileProvider.gd  
BakedTileProvider.gd  
CellTileProvider.gd  
IrregularCellProvider.gd  
RiverTileProvider.gd
```

Правильная схема:

```
фоновый поток -> Image  
главный поток -> ImageTexture
```

Это важно, потому что `Texture2D` / `ImageTexture` лучше создавать в главном потоке, а тяжёлую подготовку картинки можно делать в фоне.

2.5. Статичные слои уже начали запекаться

Это очень важное решение.

Уже есть:

```
BakedTileProvider.gd  
bake_continents_tiles.py  
bake_land_sea_tiles.py  
assets/tiles_bundle/...
```

И это правильный путь для:

```
суша/море;  
континенты;  
реальные области;  
клетки;  
провинции;  
регионы;  
страны;  
постоянные границы.
```

Статичный слой должен быть:

```
офлайн-генерация -> PNG/WebP tile bundle -> быстрый показ в игре
```

А не:

```
игрок двигает камеру -> Godot каждый раз заново рендерит геометрию
```

2.6. Данные клетки отделены от рендера

`Cell.gd` — это `Resource` с игровыми данными, а геометрия/отрисовка живёт отдельно.

Это правильно:

```
Cell.gd — данные  
IrregularCellProvider.gd — картинка  
CellCatalog.gd — загрузка данных  
TileMapView.gd — показ слоёв
```

Это значит, что ты не смешал игровую логику с визуалом. Для стратегии это критически важно.

3. Исправленная оценка IrregularCellProvider

Раньше я выделил это как риск:

```
for cell in _cells:
```

Там действительно два прохода:

1. заливка клеток;
2. отрисовка границ.

Но с учётом твоего уточнения:

этот слой будет запечён потом

это больше не критический runtime-риск.

Правильная оценка:

`IrregularCellProvider` – допустим как `tool/debug/bake-provider`.
`IrregularCellProvider` – нежелателен как постоянный `runtime-provider` для всей планеты.

То есть сам код не плохой. Он просто должен использоваться в правильном месте.

Нормально

`IrregularCellProvider` рендерит слой в разработке.
Проверяем качество границ.
Потом Python/Godot tool запекает тайлы.
В игре используется `BakedTileProvider`.

Плохо

Игрок играет.
Включён слой всех реальных областей мира.
Каждый новый тайл live-рендерится через `IrregularCellProvider`.
Каждый тайл перебирает все полигоны.

Если ты точно идёшь по схеме запекания — всё нормально.

4. Исправленная оценка `raster_px = 1024`

`raster_px = 1024` тоже не является проблемой само по себе.

Оно опасно только в таком сценарии:

runtime;
весь мир;

```
много видимых тайлов;  
панорамирование;  
каждый тайл live-рендерится в 1024 рх.
```

Но если это используется для запекания:

```
один раз сгенерировать красивые чёткие границы;  
сохранить в tile bundle;  
в игре читать готовые картинки;
```

то 1024 — нормальное решение.

Да, цена выше:

```
больше время запекания;  
больше размер ассетов;  
больше временная память при генерации;  
больше вес тайлов на диске.
```

Но это цена офлайн-качества, а не runtime-проблема.

Правильная формулировка:

```
1024px допустим для bake/tool-рендера.  
Для live runtime-слоёв по всей планете лучше 256/512 или BakedTileProvider.
```

5. Реальные runtime-риски, которые остаются

5.1. Нет лимита RAM/GPU-кэша тайлов

Это главный реальный риск.

Почти везде есть словари:

```
var _cache: Dictionary = {}  
var _tex: Dictionary = {}
```

Они есть в:

```
OnlineTileProvider.gd  
BakedTileProvider.gd  
CellTileProvider.gd
```

```
IrregularCellProvider.gd  
RiverTileProvider.gd  
SeaBorderTileProvider.gd  
TileProvider.gd
```

Проблема:

```
игрок летал по Европе;  
потом по Америке;  
потом по Азии;  
потом включал/выключал слои;  
спрайты старых тайлов удалились;  
но Texture2D остались в словарях провайдеров.
```

`queue_free()` у спрайта не освобождает текстуру, если она всё ещё лежит в `_cache` или `_tex`.

Что нужно сделать позже:

```
LRU-кэш тайлов;  
лимит на количество тайлов в памяти;  
очистка старых тайлов по last_used;  
разный лимит для разных слоёв.
```

Примерно:

```
базовая карта: 1000–2000 тайлов в RAM;  
оверлеи: 300–800 тайлов на слой;  
debug/bake layers: можно меньше;  
пустые тайлы: не хранить бесконечно.
```

Это важнее, чем оптимизировать `for cell in _cells`.

5.2. BakedTileProvider тоже может раздувать кэш

В `BakedTileProvider.gd` для отсутствующих тайлов делается:

```
_cache[key] = _blank_tex
```

Сама `_blank_tex` общая — это хорошо. Но ключи всё равно копятся в словаре.

То есть если игрок пролетел над океаном на разных LOD, в `_cache` появится много записей вида:

```
z/x/y -> blank_tex
```

Это не катастрофа, но при долгой игре словарь будет расти.

Решение то же:

```
LRU или лимит кэша.
```

5.3. TileMapView пересчитывает видимые тайлы каждый кадр

Сейчас в `_process()` каждый кадр:

```
считается LOD;  
считается видимый прямоугольник;  
собирается needed Dictionary;  
проходятся все видимые тайлы;  
проходятся все active.keys();  
парсятся строковые ключи.
```

Для текущего прототипа это нормально.

Но в будущем лучше обновлять набор тайлов только когда изменилось одно из условий:

```
камера перешла границу тайла;  
изменился LOD;  
изменился zoom достаточно сильно;  
включили/выключили слой;  
изменился размер окна;  
изменился TILE_PAD/границы карты.
```

То есть не надо каждый кадр заново решать одну и ту же задачу, если камера просто плавно едет внутри тех же тайлов.

Правильный будущий вариант:

```
TileMapView._process()  
-> камера движется визуальнo каждый кадр  
-> но _update_visible_tiles() вызывается только при смене tile-range/lod/  
layer-state
```

Это не срочно, но это хороший следующий уровень оптимизации.

5.4. Строковые ключи тайлов и split/substr в цикле

Сейчас ключи выглядят так:

```
var key := "%d|%d/%d/%d" % [li, lod, tx, ty]
```

А при выгрузке обратно парсятся:

```
var sep := (key as String).find("|")
var li := int((key as String).substr(0, sep))
var rest := (key as String).substr(sep + 1).split("/")
```

Это работает, но для горячего цикла не идеально.

Проблема не в том, что это “ужасно медленно”. Проблема в том, что это создаёт лишние строки, массивы и мелкие аллокации каждый кадр.

Лучше позже сделать одно из двух:

Вариант А

Хранить метаданные рядом со спрайтом:

```
_active[key] = {
  "sprite": spr,
  "layer": li,
  "z": lod,
  "x": tx,
  "y": ty
}
```

Вариант Б

Использовать числовой packed-key:

```
key = pack(layer, z, x, y)
```

Для текущей стадии можно оставить как есть. Но перед большой картой это стоит убрать.

5.5. MAX_GEN_PER_FRAME ограничивает не всё

В `TileMapView.gd` есть:

```
const MAX_GEN_PER_FRAME := 16
```

Но `gens` увеличивается только когда текстура уже готова и создан спрайт:

```
if tex != null:  
    _active[key] = _make_tile(...)  
    gens += 1
```

Если `provider.request_tile()` возвращает `null`, потому что тайл ещё рендерится/декодируется/качается, `gens` не увеличивается.

То есть в одном кадре можно запустить много запросов к провайдерам, особенно если включено много слоёв.

Для текущей видимой области это терпимо. Но правильнее потом разделить:

```
MAX_TILE_REQUESTS_PER_FRAME  
MAX_TILE_SPRITES_PER_FRAME  
MAX_TEXTURE_CREATES_PER_FRAME
```

Потому что это разные нагрузки:

```
request_tile() – может запустить поток/загрузку;  
ImageTexture.create_from_image() – создаёт GPU-текстуру;  
_make_tile() – создаёт Sprite2D.
```

Сейчас ограничен только последний пункт.

5.6. Провайдеры могут создать слишком много текстур за один кадр

В провайдерах есть такой паттерн:

```
for item in done:  
    _cache[key] = ImageTexture.create_from_image(img)
```

Если за один кадр вернулось много фоновых задач, главный поток создаст много текстур сразу.

Это может дать микрофриз.

Лучше добавить бюджет:

создавать не больше 8-16 ImageTexture за кадр на провайдер или общий бюджет на все провайдеры.

Это особенно важно для:

```
BakedTileProvider  
OnlineTileProvider  
CellTileProvider  
IrregularCellProvider  
RiverTileProvider
```

5.7. SeaBorderTileProvider рендерит синхронно

В `SeaBorderTileProvider.gd`:

```
var t := _render(z, x, y)  
_tex[key] = t  
return t
```

То есть рендер морских границ идёт прямо в `request_tile()`.

Если морских линий немного — нормально. Но архитектурно это слабее, чем у остальных провайдеров.

Лучше один из вариантов:

1. запечь морские границы в `BakedTileProvider`;
2. перевести `SeaBorderTileProvider` на `ThreadPool`;
3. оставить как `debug/dev`-слой, если он лёгкий.

Если слой “Моря” будет постоянным игровым слоем, я бы его запёк.

5.8. Старые фоновые задачи могут досчитываться уже ненужными

Если игрок резко увёл камеру, старые задачи всё равно завершатся и положат текстуры в кэш.

Это не баг, но лишняя работа.

Позже можно добавить:

```
view_generation_id;  
request_generation;  
проверку "тайл всё ещё нужен?";  
отбрасывание устаревших done_images.
```

Это не первая задача, но полезно для плавности.

6. Что НЕ является проблемой сейчас

6.1. Не надо срочно оптимизировать IrregularCellProvider spatial-index

Если слой будет запечён, spatial index внутри IrregularCellProvider сейчас не обязателен.

Раньше я бы предложил:

```
tile/bin -> список клеток
```

Но с твоим уточнением это уже не приоритет.

Нужно делать spatial index только если:

```
запекание стало слишком долгим;  
или ты решил оставить live-render для большого слоя;  
или нужен быстрый клик/поиск клетки под курсором.
```

Для игрового клика spatial index всё равно потом понадобится, но уже не для рендера, а для логики выбора клетки.

6.2. Не надо снижать raster_px = 1024, если это bake-качество

Для тестовых клеток и запекания границ 1024 допустим.

Снижать надо только если:

```
запекание стало слишком долгим;  
ассеты стали слишком тяжёлыми;  
видеопамять растёт из-за live-показа 1024-тайлов;  
этот слой случайно остался runtime-live.
```

6.3. Не надо переходить на C# из-за этих проблем

Все главные проблемы здесь архитектурные:

```
кэш;  
лимит создания текстур;  
запекание слоёв;  
частота обновления тайлов;  
разделение runtime/tool.
```

C# сам по себе не исправит:

```
бесконечный кэш;  
строковые ключи;  
live-рендер статичных слоёв;  
создание 50 текстур в одном кадре;  
полный пересчёт _process каждый кадр.
```

Поэтому текущий вывод по стеку остаётся:

```
оставаться на GDScript;  
оптимизировать архитектуру;  
C#/GDExtension рассматривать позже только для конкретных тяжёлых систем.
```

7. Главная архитектурная правка: разделить Runtime и Bake/Debug providers

Я бы прямо зафиксировал это в проекте.

Runtime providers

Это то, что можно использовать во время обычной игры:

```
BakedTileProvider  
OnlineTileProvider / bundle satellite tiles  
лёгкие overlay providers  
подсветка выбранной клетки  
подсветка маршрута  
туман войны  
иконки городов/армий
```

Bake / Debug providers

Это то, что можно использовать в разработке, предпросмотре или запекании:

```
IrregularCellProvider
CellTileProvider, если это старый процедурный слой
RiverTileProvider, если слой рек ещё не запечён
SeaBorderTileProvider, если рендерит синхронно
старые land_cells / test layers
```

Правило:

Тяжёлый геометрический provider не должен быть обязательным runtime-слоем релизной игры.

Идеально:

```
debug/dev режим:
    IrregularCellProvider показывает слой live

game/runtime режим:
    BakedTileProvider показывает готовый tile bundle
```

8. Конкретно по слоям

Спутник / базовая карта

`OnlineTileProvider.gd` написан нормально:

```
+ есть HTTP pool;
+ есть disk cache;
+ есть bundle cache;
+ декодирование в фоне;
+ текста создаётся в главном потоке.
```

Что улучшить позже:

```
- LRU-кэш в памяти;
- приоритет видимых запросов над старыми;
- лимит создания текстур за кадр.
```

Континенты

Сейчас сделано правильно:

```
континенты уже идут через BakedTileProvider, если есть tiles_bundle.
```

Это хороший пример того, как должны работать все статичные крупные слои.

Суша/море

Тоже правильно:

```
land_sea_baked через BakedTileProvider;  
land_sea.json остаётся источником логики.
```

Это идеальная схема:

```
картинка отдельно;  
игровые данные отдельно.
```

Реальные области admin1

Сейчас они ещё подключаются через:

```
IrregularCellProvider.new("res://assets/admin1_areas.json", ...)
```

С учётом твоего плана это нормально как временный слой.

Но перед нормальной игрой его лучше перевести в:

```
real_areas_baked / provinces_baked / admin_areas_baked
```

Название отдельно надо привести к человеческому виду, потому что `admin1` — GIS-термин, а не нормальное игровое имя.

Итог:

```
Сейчас допустимо.  
В релизном runtime лучше запечь.
```

Тестовые клетки Ла-Коруньи

Там `raster_px = 1024` абсолютно нормально.

Причина:

маленькая область;
1–2 тайла;
цель — проверить качество клеток и границ.

Оптимизировать это сейчас не надо.

Реки

`RiverTileProvider` уже работает через `WorkerThreadPool`, это хорошо.

Но реки — статичный слой. Поэтому в будущем лучше:

или запечь реки;
или оставить `live async`, если их мало и они не тормозят.

Для глобальной стратегии я бы в итоге запёк:

крупные реки;
береговые линии;
морские границы;
административные границы;
континенты;
суша/море.

A dynamic overlay оставить только для того, что реально меняется.

Моря / морские границы

`SeaBorderTileProvider` сейчас синхронный. Это самый подозрительный слой из визуальных.

Если слой включается редко и лёгкий — можно оставить.

Но если он будет частью обычной карты, лучше:

```
запечь;  
или перевести на WorkerThreadPool.
```

9. Игровые данные клеток

`Cell.gd` сейчас нормален для MVP.

Плюсы:

```
+ чистый Resource;  
+ нет Node;  
+ нет геометрии внутри Cell;  
+ есть понятные поля;  
+ есть связь с диздоками;  
+ есть CellCatalog;  
+ есть GameplayTables.
```

Но для будущей массовой симуляции 20 000+ клеток лучше не гонять всё через объекты и строки.

Сейчас в `Cell.gd` много строковых типов:

```
var surface_type: String  
var relief_type: String  
var natural_cover_type: String  
var soil_type: String  
var climate_type: String  
var development_type: String
```

Для читаемости это хорошо.

Для тяжёлой симуляции лучше потом сделать так:

```
Cell.gd / CellCatalog.gd:  
    удобные данные для загрузки, UI, редактора, отладки  
  
WorldRuntime.gd:  
    PackedInt32Array / PackedFloat32Array / PackedByteArray  
    быстрые массивы по cell_id для годовой симуляции
```

То есть:

```
cell.development_type == "vineyards"
```

в будущем лучше заменить в симуляторе на:

```
development_type[cell_id] == DEVELOPMENT_VINEYARDS
```

Но это не надо делать прямо сейчас. Сейчас ещё нет полной симуляции.

Правильный момент для этого:

```
когда появится годовой тик экономики/населения/освоения хотя бы на 5000–20000  
клеток.
```

10. Что надо сделать в первую очередь

Приоритет 1 — добавить лимит кэша тайлов

Это самый важный реальный пункт.

Нужно сделать общий подход:

```
каждый provider хранит не бесконечный _cache/_tex,  
а ограниченный LRU-кэш.
```

Минимальный вариант:

```
key -> texture  
key -> last_used_frame  
если cache.size() > limit:  
    удалить самые старые
```

Это защитит от роста памяти при долгой игре.

Приоритет 2 — запечь все постоянные тяжёлые polygon-слои

Обязательные кандидаты:

```
реальные области;  
суша/море;  
континенты;  
крупные реки;  
морские границы;
```

будущие клетки;
будущие провинции/регионы/страны.

В runtime должны остаться:

BakedTileProvider + игровые данные для клика/логики.

А не live-рендер огромных полигонов.

Приоритет 3 — добавить бюджет создания Texture2D за кадр

Сейчас провайдер может за один кадр создать все готовые текстуры из `done`.

Лучше:

`MAX_TEXTURE_CREATES_PER_FRAME = 8` или `16`

И обрабатывать `done_images` постепенно.

Это снизит микрофризы.

Приоритет 4 — разделить лимит запросов и лимит спрайтов

Сейчас есть:

`MAX_GEN_PER_FRAME`

Но лучше иметь:

`MAX_TILE_REQUESTS_PER_FRAME`
`MAX_TILE_SPRITES_PER_FRAME`
`MAX_TEXTURE_CREATES_PER_FRAME`

Потому что это разные типы нагрузки.

Приоритет 5 — оптимизировать TileMapView._process

Не срочно, но желательно.

Сейчас можно оставить. Потом:

```
не пересобирать needed каждый кадр;  
обновлять тайлы только при смене tile_range/lod/layer visibility;  
убрать split/substr из горячего цикла.
```

Приоритет 6 — привести имена файлов к смысловым

Это не FPS-оптимизация, но это оптимизация разработки.

Например, `admin1_areas.json` — технически понятно GIS-разработчику, но плохо для проекта игры.

Лучше что-то вроде:

```
real_areas.json  
real_provinces.json  
world_administrative_areas.json
```

Точное имя надо выбрать по смыслу слоя.

11. Что сейчас НЕ надо делать

Не надо сейчас:

- переписывать проект на C#;
- переписывать TileMapView с нуля;
- оптимизировать IrregularCellProvider spatial-index, если слой будет запекаться;
- снижать raster_px=1024 для тестовых/bake-слоёв;
- делать сложную ECS-архитектуру;
- заранее переводить все Cell в массивы, пока нет годовой симуляции;
- делать mesh-рендер клеток вместо tile bundle;
- заводить огромную инфраструктуру под AI/экономику до появления MVP.

Текущая стратегия правильная:

```
сначала довести карту и данные до рабочего состояния;  
потом запечь статичные слои;  
потом добавить клик/выбор клетки;  
потом город/годовой тик;  
потом профилировать.
```

12. Обновлённый итоговый вердикт

Код можно считать **хорошо прописанным для текущего этапа**.

Он уже следует главным правилам оптимизированной карты:

```
не Node на клетку;  
тайлы вместо гигантской картинки;  
LOD;  
фоновые задачи;  
дисковый кэш;  
запечённые статичные слои;  
данные отдельно от рендера.
```

Два пункта, которые раньше выглядели опасными:

```
for cell in _cells;  
  raster_px = 1024;
```

после твоего уточнения считаются допустимыми, потому что это цена bake/tool-слоя, а не финального runtime.

Главные реальные проблемы теперь другие:

1. бесконечный RAM/GPU-кэш текстур;
2. отсутствие бюджета на массовое создание ImageTexture за кадр;
3. TileMapView пересчитывает видимые тайлы каждый кадр;
4. строковые ключи и split/substr в горячем цикле;
5. некоторые статичные слои ещё live/debug, а должны стать baked;
6. будущая симуляция клеток пока не отделена в быстрый WorldRuntime.

Финальная оценка:

```
Для текущего прототипа — хорошо.  
Для карты мира после запекания слоёв — почти хорошо.  
Для полной игры с экономикой, AI, логистикой и 20 000+ клеток — нужно будет  
отдельно строить runtime-симулятор на компактных массивах.
```

Главное решение:

```
Не переписывать.  
Не переходить на C#.  
Не оптимизировать debug/bake-слои раньше времени.
```

Сначала добавить LRU-кэш, запекание всех статичных слоёв и бюджет создания текстур.